

DESAFIO TÉCNICO – ENGENHARIA DE QUALIDADE QA / SDET

HCXpert – Especificação revisada e orientada a qualidade de produto

Versão 2.0 • 2026 • Documento de avaliação técnica

Objetivo: avaliar raciocínio de qualidade, engenharia de testes, automação sustentável, observabilidade e capacidade de comunicar riscos — não apenas a quantidade de scripts produzidos.

1. Objetivo e princípios de avaliação

Este desafio deve ser tratado como uma pequena entrega de engenharia de qualidade de produto. O candidato deverá transformar requisitos incompletos em uma estratégia verificável, explicitar hipóteses, construir uma automação sustentável e demonstrar como os testes seriam executados com segurança em uma esteira de integração contínua. A avaliação considera a qualidade das decisões, a capacidade de detectar riscos e a manutenção futura, e não exige cobertura artificial de todos os caminhos possíveis.

Quando o site ou uma API externa apresentar indisponibilidade, comportamento inconsistente ou mudança de contrato, o candidato não deve mascarar a falha. Deve registrar o fato, classificar o impacto, separar defeito do produto de falha de ambiente e propor uma alternativa reproduzível. Esse comportamento é parte essencial da avaliação SDET.

Princípio	Aplicação esperada
Risco antes de volume	Priorizar fluxos críticos, regras de negócio e falhas de alto impacto.
Testabilidade explícita	Identificar dependências, dados, seletores, contratos e limitações do ambiente.
Automação sustentável	Código legível, desacoplado, determinístico, observável e fácil de evoluir.
Evidência reproduzível	Cada falha deve conter contexto, passo, resultado esperado/obtido, logs e evidência.
Feedback rápido	Separar smoke, regressão, API e cenários dependentes de ambiente.
Qualidade contínua	Incluir execução local, CI, lint, relatório, artefatos e critérios de saída.

2. Escopo funcional e não funcional

O sistema sob teste será o Automation Exercise, utilizando a URL base configurável por variável de ambiente. O escopo obrigatório é composto por autenticação, busca de produtos, inclusão no carrinho, checkout, validação de campos obrigatórios e duas integrações de API. O candidato deve declarar no README quais comportamentos foram observados diretamente, quais foram inferidos e quais não puderam ser validados por limitação do ambiente.

2.1 Funcionalidades obrigatórias

ID	Funcionalidade	Critérios mínimos de aceite
WEB-01	Login	Credenciais válidas permitem acesso; credenciais inválidas não autenticam e exibem mensagem verificável; sessão e logout são tratados.
WEB-02	Busca	Resultado correspondente é exibido; busca sem correspondência comunica o estado; entrada vazia e normalização de espaços são avaliadas.
WEB-03	Carrinho	Produto, preço, quantidade e subtotal são consistentes entre listagem, detalhe e carrinho; inclusão duplicada possui comportamento documentado.
WEB-04	Checkout	Usuário autenticado consegue preencher dados obrigatórios, revisar pedido e concluir o fluxo quando o ambiente permitir.
WEB-05	Validação	Campos obrigatórios e dados de pagamento inválidos bloqueiam avanço com mensagens associadas ao campo; mensagens não devem depender apenas de cor.
API-01	GET Trello	Endpoint, contrato de resposta, status esperado e eventual autenticação devem ser configuráveis. O campo list.name deve ser validado somente após confirmar a estrutura e registrar a resposta sanitizada.
API-02	POST Automation Exercise	Criar conta com dados únicos e validar status, contrato e regra de negócio para sucesso e parâmetros ausentes/ inválidos. Deve existir estratégia de limpeza ou geração idempotente.

2.2 Fora de escopo, salvo justificativa

Performance, segurança ofensiva, compatibilidade ampla entre navegadores, acessibilidade automatizada completa e testes de contrato entre serviços não são obrigatórios para aprovação, mas o candidato deve indicar como os trataria em uma iniciativa real e quais riscos permanecem sem cobertura.

3. Perguntas em aberto e hipóteses obrigatórias

O desafio original contém dependências e decisões não especificadas. A ausência de uma resposta não deve ser ocultada: o candidato deve registrar uma hipótese no README e implementar o comportamento de forma configurável sempre que possível.

Tema	Lacuna identificada	Tratamento exigido
Ambientes	Não há ambiente de homologação, credenciais ou garantia de estabilidade.	Usar BASE_URL, credenciais e timeouts via .env; documentar disponibilidade e limitações.
Dados	Não há política para dados existentes, colisões ou limpeza.	Gerar dados únicos, evitar usuários reais, prover massa controlada e estratégia de teardown quando aplicável.
API Trello	Endpoint público pode exigir chave/token, variar ou tornar-se indisponível.	Não versionar segredos; validar configuração; classificar indisponibilidade como bloqueio de ambiente e fornecer modo de execução controlado.
Expectativas de status	O documento não define status esperados para todos os casos.	Declarar status esperado por cenário e justificar quando houver mais de um resultado aceitável.
Seletores	Não há contrato de seletores nem data-testid garantido.	Preferir atributos estáveis e semânticos; documentar seletores frágeis e encapsular sua manutenção.
Checkout/pagamento	Não está claro se há gateway real ou apenas formulário simulado.	Não usar dados financeiros reais; declarar o limite do teste e validar apenas o comportamento observável do ambiente.
CI/CD	Não há sistema de CI, matriz de browsers ou política de retries.	Entregar workflow reproduzível, comandos, artefatos e política explícita para retries sem mascarar flakiness.

4. Parte 1 – Estratégia e especificação BDD

Os cenários devem ser escritos em Gherkin declarativo, com linguagem de negócio e sem termos de implementação como describe, it, cy.get ou nomes de métodos. Cada cenário deve possuir identificador único, objetivo, pré-condições claras, dados controlados, resultado esperado e vínculo com o requisito correspondente.

- Criar ao menos um cenário positivo e um negativo por funcionalidade obrigatória; incluir cenários de fronteira quando a regra de negócio justificar.
- Evitar cenários longos e acoplados. Cada cenário deve testar uma intenção principal e ser independente de ordem de execução.
- Usar Background apenas para contexto realmente compartilhado e manter exemplos parametrizados em Scenario Outline quando reduzirem duplicação sem prejudicar leitura.
- Incluir uma matriz de rastreabilidade entre requisito, cenário, automação, evidência e resultado.
- Anexar um parecer crítico sobre testabilidade, riscos, ambiguidades e decisões tomadas.

Exemplo de cenário declarativo

```
Feature: Autenticação
  Scenario: Usuário não autenticado informa senha inválida
    Given que existe um usuário cadastrado
    When ele informa uma senha inválida
    And solicita o acesso
    Then o sistema deve rejeitar a autenticação
    And deve informar que as credenciais são inválidas
    And não deve criar uma sessão autenticada
```

5. Parte 2 – Automação web e API

A implementação deve usar TypeScript ou JavaScript com Cypress em versão estável definida no package.json. O uso de Cucumber deve ser feito por uma integração compatível e documentada; os arquivos .feature não podem conter sintaxe de frameworks xUnit/Mocha. Step definitions devem orquestrar o fluxo, enquanto page objects ou componentes encapsulam interação e seletores. A camada de API deve ser separada da camada visual e reutilizar funções de configuração, validação e geração de dados.

5.1 Requisitos de engenharia

Área	Requisito verificável
Configuração	BASE_URL, credenciais, chaves, timeouts e flags devem vir de ambiente/configuração; segredos nunca entram no repositório.
Dados	Fixtures devem conter dados não sensíveis; dados mutáveis devem ser gerados por fábrica com identificador único.
Seletores	Preferir data attributes estáveis, role/label e seletores semânticos; não depender de classes visuais ou posição CSS.
Assíncronismo	Não usar esperas fixas como estratégia principal; sincronizar por estado, requisição, URL ou elemento observável.
Assertions	Validar resultado de negócio, contrato e invariantes; evitar asserts genéricos que apenas confirmem existência da página.
Reuso	Criar comandos ou helpers pequenos, coesos e documentados; evitar comandos mágicos que escondam muitas decisões.
Isolamento	Cada cenário deve preparar seu estado e não depender de execução anterior; limpar ou renovar dados quando necessário.
Flakiness	Retries devem ser limitados e apenas em CI, acompanhados de relatório de instabilidade; retry não pode substituir correção.
API	Validar status, headers relevantes, esquema/estrutura e regra de negócio. Registrar request/response sanitizados em falha.
Observabilidade	Gerar screenshots e vídeo quando aplicável, logs legíveis, relatório Cucumber e artefatos por execução.

5.2 Estrutura de referência

```
.
├── cypress/e2e/features/
├── cypress/e2e/step_definitions/
├── cypress/support/page_objects/
├── cypress/support/commands/
├── cypress/support/factories/
├── cypress/fixtures/
├── cypress/schemas/
├── cypress/evidencias/
├── scripts/
├── .github/workflows/
├── cypress.config.ts
├── package.json
├── .env.example
├── README.md
└── TRACEABILITY.md
```

A estrutura acima é uma referência mínima, não uma restrição artificial. O candidato pode adaptar nomes e agrupamentos se explicar a decisão. A organização será avaliada pelo princípio de coesão, legibilidade e manutenção, e não por correspondência textual de diretórios.

6. Execução, CI e qualidade do código

O projeto deve ser executável por uma pessoa que não participou da implementação. O README deve conter pré-requisitos, instalação limpa, configuração de ambiente, comandos de execução por suíte, execução headless, geração de relatório, localização de artefatos e troubleshooting. Deve existir pelo menos uma execução automatizada em CI, com cache seguro de dependências, publicação de resultados e falha do job quando testes obrigatórios falharem.

Comando esperado	Finalidade
npm ci	Instalação reprodutível baseada no lockfile.
npm run lint	Verificação estática e padronização.
npm run test:smoke	Fluxo rápido de sanidade.
npm run test:e2e	Suíte web completa em modo headless.
npm run test:api	Suíte de serviços.
npm run report	Consolidação do relatório e evidências.

7. Evidências, triagem e relatório

Para cada falha, o relatório deve distinguir falha de produto, falha de teste, falha de dados e indisponibilidade de ambiente. A evidência deve permitir reprodução sem interpretação subjetiva. O relatório final deve apresentar o status de cada requisito, cobertura executada, cenários bloqueados, limitações conhecidas e riscos residuais.

Campo mínimo da evidência	Conteúdo
Identificação	ID do requisito, feature, cenário, execução e commit.
Contexto	Ambiente, browser, versão, data/hora e configuração relevante.
Reprodução	Pré-condições e passos objetivos.
Resultado	Esperado versus obtido, com status e classificação da falha.
Artefatos	Screenshot/vídeo/log/response sanitizado e link no relatório.
Ação	Severidade, hipótese de causa, recomendação e responsável sugerido.

8. Critérios de aceite e rubrica de avaliação

A aprovação exige que a entrega seja instalável, executável e tecnicamente justificável. Um projeto visualmente completo, mas não determinístico, sem rastreabilidade ou que esconda falhas de ambiente, não deve receber avaliação máxima.

Dimensão	Peso	Evidência de excelência
Estratégia e risco	20%	Escopo priorizado, hipóteses explícitas, cobertura de positivos/negativos/fronteiras e parecer crítico.
BDD e rastreabilidade	15%	Gherkin declarativo, legível, independente e matriz requisito–cenário–teste–evidência.
Arquitetura de automação	20%	Separação de responsabilidades, seletores sustentáveis, baixo acoplamento e reuso consciente.
Robustez e dados	15%	Isolamento, dados únicos, sincronização por estado, baixo flake e tratamento de indisponibilidade.
API e contratos	10%	Status, estrutura, headers/regras de negócio, sanitização e cenários de erro.
CI/CD e observabilidade	10%	Pipeline reproduzível, lint, relatórios, artefatos e política transparente de retry.
Documentação e comunicação	10%	README operacional, limitações, decisões, triagem e relatório executivo.

Critérios eliminatórios

- Falha na instalação ou execução sem instruções adicionais não documentadas.
- Segredos, tokens ou dados pessoais/financeiros reais versionados.
- Testes que dependem da ordem de execução ou de esperas fixas para funcionar.
- Automação que reporta sucesso sem validar resultado de negócio.
- Falhas conhecidas omitidas do relatório ou mascaradas por retries ilimitados.
- Entrega sem evidência mínima para os fluxos obrigatórios.

9. Entregáveis

1. Repositório com código-fonte, lockfile, configuração, testes, scripts e workflow de CI.
2. Arquivo README.md com setup, execução, hipóteses, limitações, troubleshooting e decisão arquitetural.
3. Features Gherkin e step definitions com paridade verificável.
4. Matriz TRACEABILITY.md relacionando requisitos, cenários, automação, evidência e status.
5. Relatório HTML ou equivalente com resultados, evidências e classificação das falhas.
6. Parecer crítico SDET sobre testabilidade, riscos residuais e próximos incrementos.

A entrega deve ser feita por link de repositório ou arquivo compactado conforme orientação da equipe. O repositório deve ser reproduzível por um avaliador externo, sem depender de credenciais pessoais do candidato.

10. Checklist final do candidato

- ☐ Instalação limpa executada em ambiente novo.
- ☐ Todas as variáveis necessárias descritas no .env.example.
- ☐ Nenhum segredo ou dado real no histórico do repositório.
- ☐ Cenários positivos, negativos e de fronteira justificados.
- ☐ Testes web e API isolados e executáveis separadamente.
- ☐ Status, contratos e regras de negócio validados.
- ☐ Screenshots, logs e relatórios gerados em falhas.
- ☐ CI executada com artefatos e falha correta do job.
- ☐ Limitações e bloqueios externos registrados.
- ☐ Matriz de rastreabilidade revisada antes da entrega.

Referências técnicas

- [1] Cypress Documentation — Best Practices: <https://docs.cypress.io/app/core-concepts/best-practices>
- [2] Cypress Documentation — Network Requests: <https://docs.cypress.io/api/commands/request>
- [3] Cucumber Documentation — Gherkin Reference: <https://cucumber.io/docs/gherkin/reference/>
- [4] OWASP — API Security Top 10: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [5] GitHub Actions Documentation — Artifacts: <https://docs.github.com/en/actions/using-workflows/storing-workflow-data-as-artifacts>